```
end
```

produces:

```
Run options: --seed 51235
# Running:

...
Finished in 0.000462s, 6493.5063 runs/s, 99567.0972 assertions/s.
3 runs, 46 assertions, 0 failures, 0 errors, 0 skips
```

Inside the teardown method, you can detect whether the preceding test succeeded with the passed? method.

Creating Mock Objects in Minitest

Minitest has the ability to allow you to create *mock objects*, which are objects that simulate the API of an existing object in the system, typically providing a canned response instead of a more expensive or fragile real response. A minitest mock object can be *verified*, meaning it will raise a failure if methods you expected to be called were not called during the test. Using these mock object expectations allows for a style of testing where instead of testing the result of a method by verifying its output, you test the behavior of the method by verifying that it makes expected calls to other methods.

In minitest, a mock object is created like any other Ruby object; you then add the methods you wish the mock to respond to via the expect method. At the end, you can optionally test that all expected methods were called with verify.

For example, we can re-write our playlist builder test so that we don't need to create a "real" DBI instance (the word real is in scare quotes because for this contrived example, even the DBI instance in the previous code was faked…). Behind the scenes, our playlist builder calls connect and disconnect on the DBI instance. We can instead create a mock object:

unittesting/test_playlist_builder_mock.rb

```
require "minitest/autorun"
require_relative "playlist_builder"

class TestPlaylistBuilder < Minitest::Test
  def setup
    @database = Minitest::Mock.new
    @database.expect(:connect, true)
    @database.expect(:disconnect, false)
    @playlist_builder = PlaylistBuilder.new(@database)
  end

  def teardown
    @database.disconnect
    @database.verify
  end

  def test_empty_playlist
    assert_empty(@playlist_builder.playlist)
  end

  def test_artist_playlist
    @playlist_builder.include_artist("krauss")
    refute_empty(@playlist_builder.playlist, "Playlist shouldn't be empty")
    @playlist_builder.playlist.each do |entry|
```